

Individual Submission

Cole Dorman

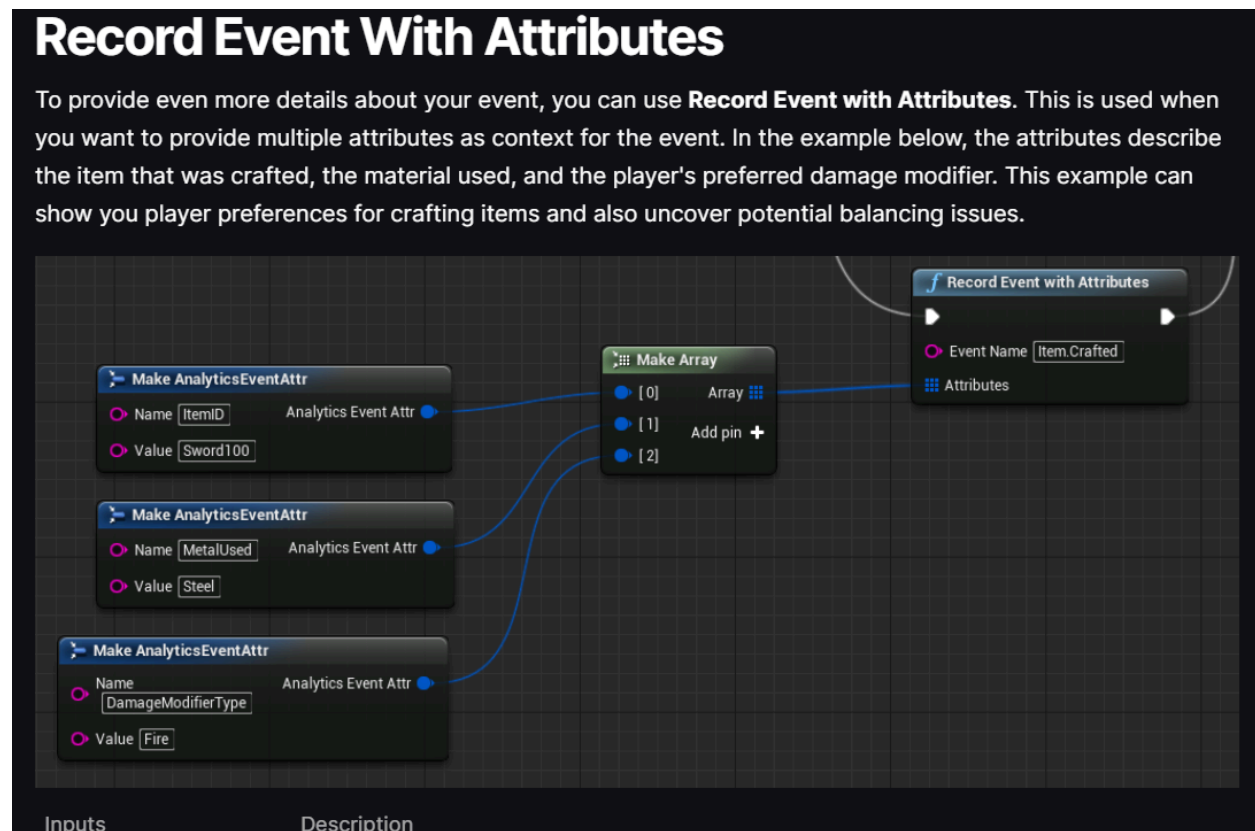
Initial Research

To start on this project I began by firstly researching online to see what unreal engine 5 had built in to handle telemetry, I found the following:

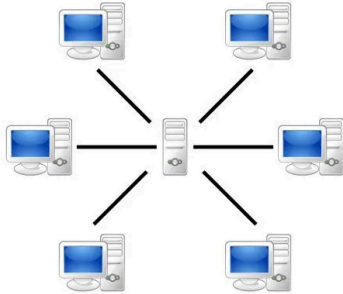
[Unreal Engine Blueprint Analytics Plugin](#)

[Unreal Insights](#)

The Blueprint Analytics plugin turned out to not have the functionality and flexibility I needed, and Unreal Insights was primarily for performance, so it was not directly useful. The image below is an example of why I had thought the Analytics Plugin would be what I had needed.



Testing



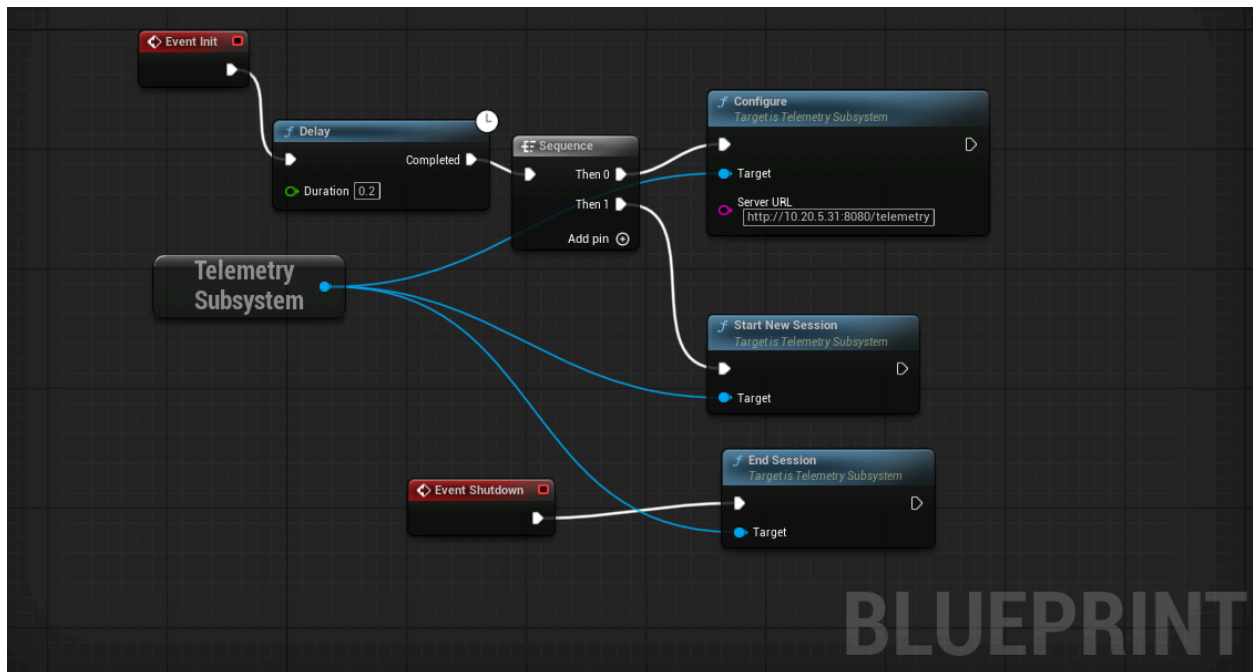
Immediately since I needed to do LAN based networking to send data from clients to the server, I had run some smaller scale testing by using a PowerShell script to simulate fake data, using a Sin wave, and then I had used a Python http server to receive the data and then store in it a file. The tests worked flawless and showed no data loss over LAN even when under pressure of large amount of requests per second (> 250). Once these succeeded I had a proof of concept I got to work creating a framework for how I would structure my telemetry system as a C++ unreal engine plugin.

Refinement

After realizing the limitations, I had consulted Ivan for direction and had taken a look at a potential C++ implementation of my own telemetry system. Since I had tested using HTTP to send json over LAN to a host from a client I started by looking into unreal engine's modules in C++. After some research I had found Json module and its helper module; JsonUtilities. Being able to utilize the reflection based serialization that unreal engine already has native, kept my code clean and reduces overhead.

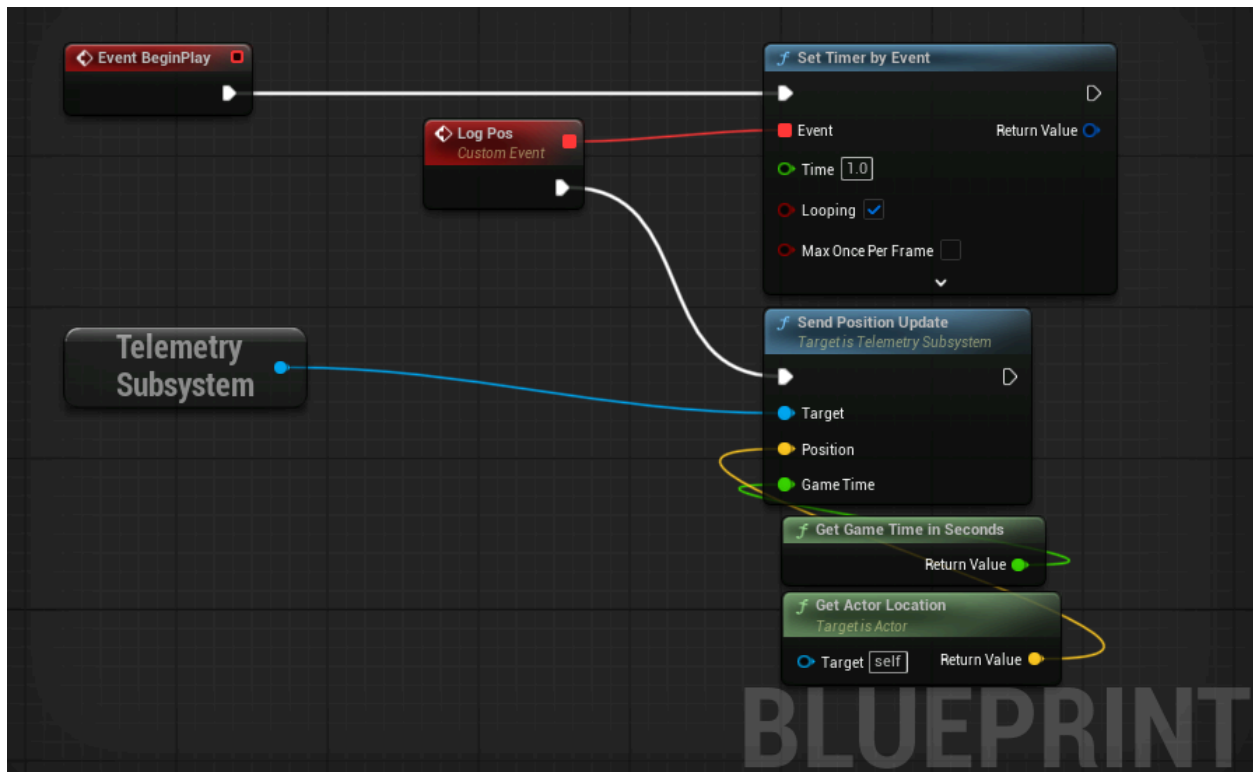
Implementation

Once the C++ plugin was created, and I had blueprint callable methods setup, this was the exact blueprint implementation I had set up for the first playable.



This is the EventGraph of the GamelInstance, which is where I handle the setup of the Telemetry Subsystem I had created.

Once I had initialized and set up the telemetry system in the game instance, it was utilized from within the Player_BP's EventGraph, via a looping timer.



This is the EventGraph of the Player, here is where I am using a timer to log the position over time. The sweet spot for the loop time I found was 0.25 seconds.

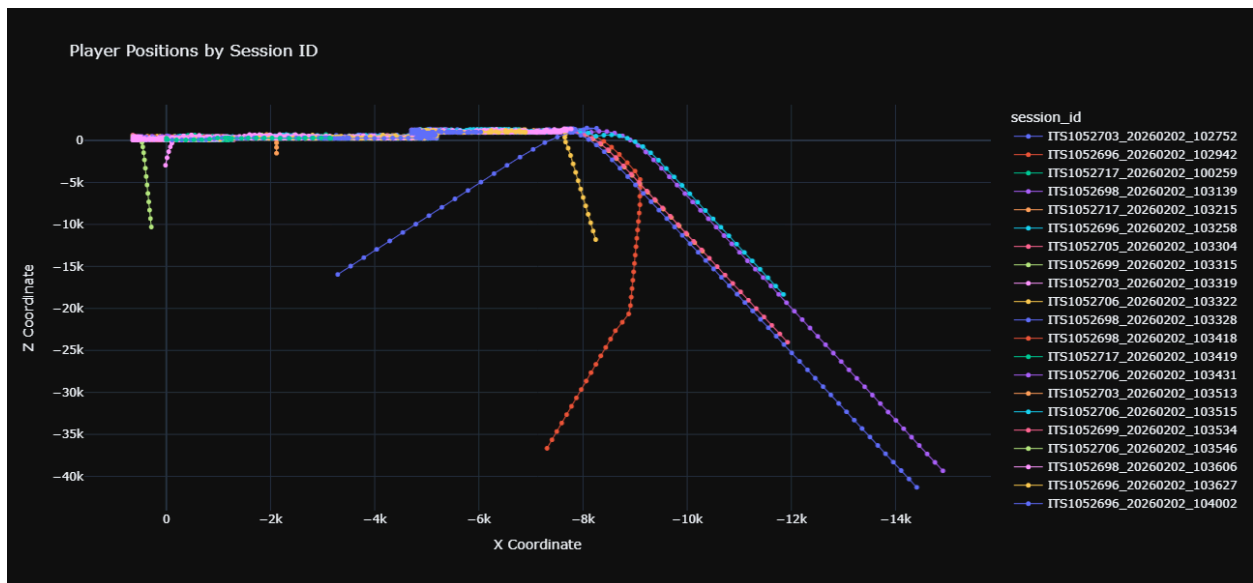
Visualization

This stage was a bit of a process for me, as I was rusty and had some refreshing to do on data analytics and visualization, as well as the fact I was using a new tech stack, UV + Marimo notebooks in python.

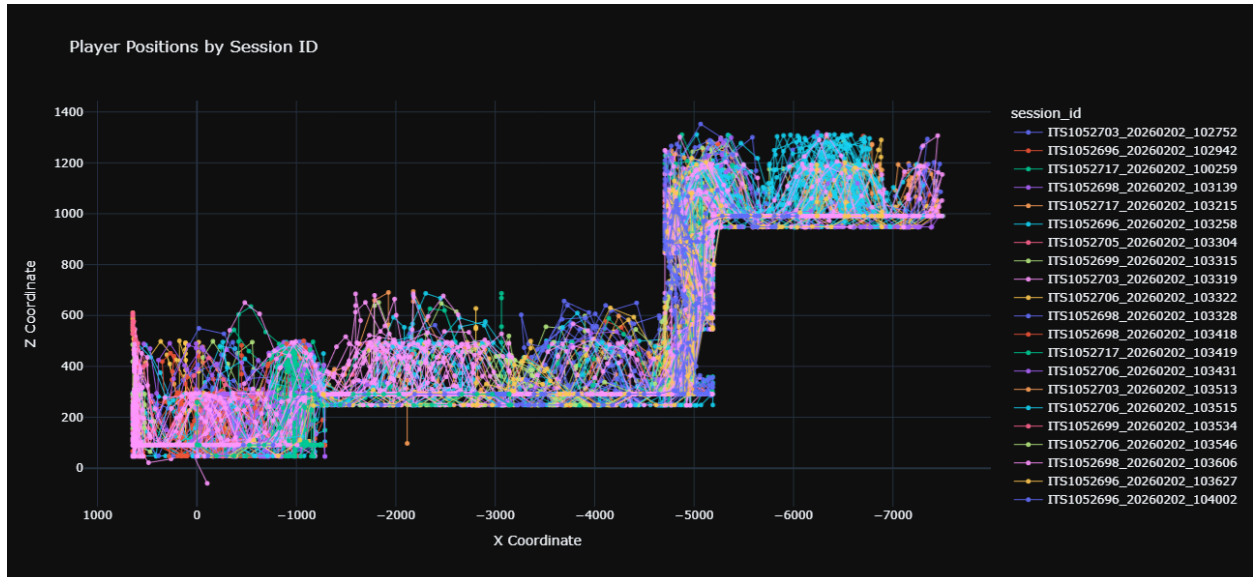
This is what the head of the raw collected data looks like:

machine_id str unique: 10	session_id str unique: 111	event_type str unique: 1	frame int64 min: 1 max: 6.377 unique: 6.377	game_time float64 min: 0.22 max: 1.59426 unique: 52.247	server_timestamp str unique: 52.882	x float64 min: -28.51142 max: 645.96 unique: 18.516	y float64 min: -456 max: 677.37 unique: 5.941	z float64 min: -105.4
ITS1052709	ITS1052709_20260201_173819	position	1	0.26292011141777	2026-02-01T17:38:19.695449	0	0	258.4
ITS1052709	ITS1052709_20260201_173819	position	2	0.51292227382659	2026-02-01T17:38:19.944710	0	0	91.14
ITS1052709	ITS1052709_20260201_173819	position	3	0.762924313545227	2026-02-01T17:38:20.194938	0	0	91.14
ITS1052709	ITS1052709_20260201_173819	position	4	1.01295530796051	2026-02-01T17:38:20.444637	-27.022756920304516	0	91.15
ITS1052709	ITS1052709_20260201_173819	position	5	1.260033249855041	2026-02-01T17:38:20.691865	-221.16123798386002	0	48.30
ITS1052709	ITS1052709_20260201_173819	position	6	1.5100358724594112	2026-02-01T17:38:20.941840	-452.3634278328834	0	108.7
ITS1052709	ITS1052709_20260201_173819	position	7	1.760053277015686	2026-02-01T17:38:21.191711	-700.9585799764296	0	239.3
ITS1052709	ITS1052709_20260201_173819	position	8	2.010083436965942	2026-02-01T17:38:21.441990	-882.707491615601	0	326.0
ITS1052709	ITS1052709_20260201_173819	position	9	2.260102272033691	2026-02-01T17:38:21.691905	-682.5149740154537	0.0000000000000000265362839018493	289.4
ITS1052709	ITS1052709_20260201_173819	position	10	2.510133266448974	2026-02-01T17:38:21.941926	-432.48387529204217	0.00000000000000005715626237574143	105.4

This is what it would look like if plotted without constraining to the level bounds and cleaning it:

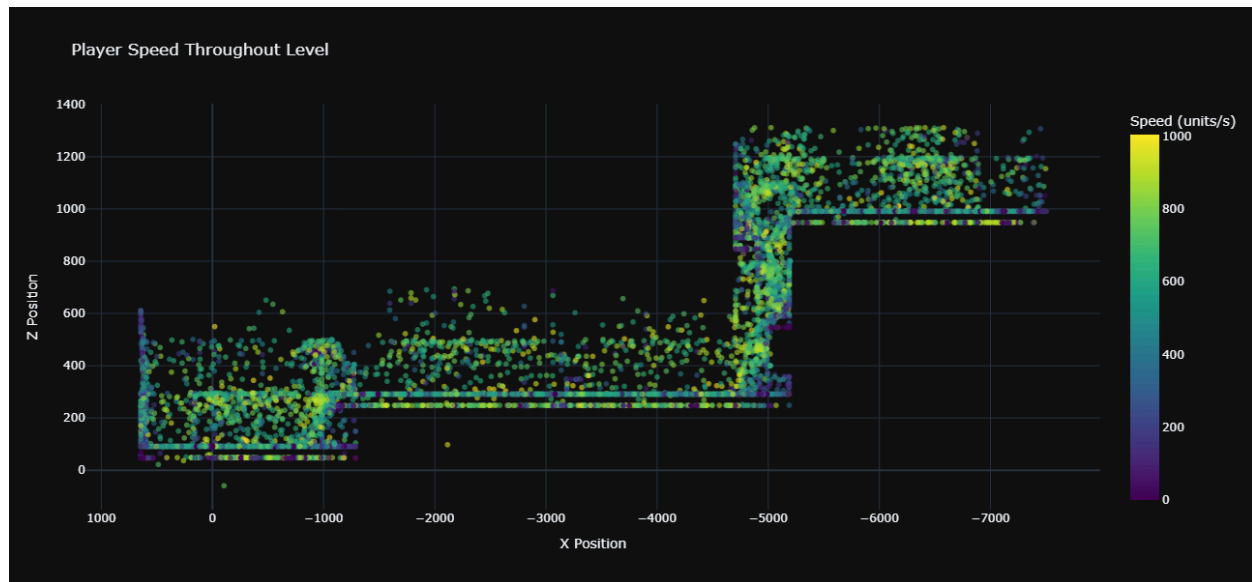


This chart below shows the cleaned and organized data:



*Lines are drawn to show the path each player took, each session_id is a unique player's path.
(Some have similar colors)*

Here is the same data except we are visualizing velocity, in order to identify areas where players tend to dwell at.



Faster is more yellow, while slow is dark purple.

Self assessment

The difficulty of the tasks I had completed varies substantially, whilst some were trivial once I had done the research, many had needed substantial amounts of research and careful planning to be successful.

The total contribution I have made comes out to around 34 hours. Four hours were spent testing LAN capability for sending JSON data through HTTP to a python server. Twelve of which were spent researching and planning implementation and integration with the engine, once this was done four hours were spent adding positional logging. Lastly seven hours were spent researching, implementing and testing different ways of visualizing the data that was collected from the first playtest.

Reflection

I feel I had met my expectations for this milestone, there was however some crunch in getting things done for the first playtest, as some of my systems interact with other people's work. I do not think I need to take on more tasks, but I also do not think I need to offload any, I am in a good spot currently.

I feel the quality of my work is quite high, as due to my extensive research on the topics before starting to code I had an excellent idea of what native tools I can use, which limits overhead and improves readability. I also have spent time adding proper keywords for the blueprint nodes as well as descriptions.

I will be taking on more tasks and one critical issue I had noticed with some of my tasks outlined on the Asset Tracker is that they did not account for multiple iterations. For example the visualizations I have now are the first pass, for next playtest I would like to try to expand on them, Ideally I want to see if I can make a heatmap work.

One of the biggest places I want to improve is coupling. Currently due to the nature of the system I am creating it has to be called from the Player_BP's EventGraph, and since other people are working there it can be challenging to get access. I am unsure how feasible it is but I would like to see if I can bind to inputs and other events from C++ without actually needing to interact with the player blueprint, as we are using a check in/check out model.